Методические указания к лабораторной работе №3

Иерархия классов, наследование, полиморфизм. Абстрактные классы, виртуальные методы. Сериализация в XML/JSON. Обобщенные объектные коллекции.

Данная работа нацелена на практическую отработку всех базовых принципов ООП, их комплексное применение в рамках одной программы на C#.

Анализируя текст задания, прежде всего, нужно выделить сущности, образующие иерархию. Например, пассажирские и грузовые самолеты – это объекты дочерних классов. Базовый класс – самолет, содержит общие для дочерних классов поля, свойства и методы.

В каждом задании у базового класса должен быть абстрактный метод, переопределяемый дочерними классами. На примере самолетов, это общий взлетный вес, рассчитанный по-разному для пассажирских и грузовых бортов.

Разработка программы включает создание класса-контейнера дочерних объектов. Например, авиакомпания содержит коллекцию самолетов. В программе обязательно должен быть алгоритм, обрабатывающий разнотипные дочерние объекты, как ссылки на базовый класс, например, расчет общего взлетного веса всех самолетов.

Рассмотрим подробно все классы. Введем перечисление **enum PlaneType** – тип самолета, по нему удобно определять вес пустого самолета, который запишем в статическую таблицу **Dictionary<PlaneType, double>** внутри базового класса **Самолет Plane**.

```
public enum PlaneType
{
    None,
    AirbusA310,
    AirbusA320,
    Boeing737,
    Boeing747
}
```

Отсутствие значения типа самолета зададим отдельным нулевым значением None.

```
/// <summary>
/// Базовый абстрактный класс 'Самолет'
/// </summary>
public abstract class Plane
{
    private static readonly Dictionary<PlaneType, double> _emptyWeights =
        new Dictionary<PlaneType, double>
            {
                [PlaneType.AirbusA310] = 82000,
                [PlaneType.AirbusA320] = 36750,
                [PlaneType.Boeing737] = 26400,
                [PlaneType.Boeing747] = 186000
            };
```

Таблица весов пустых самолетов представлена в виде закрытого статического поля класса. Статические данные всегда присутствуют во всех объектах в единственном экземпляре, поэтому

экономится память. Инициализация статических структур данных происходит до первого обращения к классу или объекту класса в т.н. статическом конструкторе класса.

Далее добавим в класс поля, конструкторы, свойства и методы, общие для всех самолетов. Базовый класс возьмет на себя ответственность по заполнению поля текущего пустого веса самолета исходя из его типа и табличного значения.

```
    protected PlaneType _planeType;
    protected double _emptyWeight;

    protected Plane() { }

    protected Plane(PlaneType type, string number)
    {
        Type = type;
        Number = number;
    }

    /// <summary>
    /// Тип самолета (определяет вес без груза)
    /// </summary>
    public PlaneType Type
    {
        get { return _planeType; }
        set
        {
            _planeType = value;
            _emptyWeight = _emptyWeights[_planeType];
        }
    }

    /// <summary>
    /// Серийный номер
    /// </summary>
    public string Number { get; set; }

    /// <summary>
    /// Взлетный вес (переопределяется в наследниках)
    /// </summary>
    public abstract double TakeoffWeight { get; }
}
```

Закрытые поля, которые должны быть доступны в дочерних классах, обозначаются областью видимости protected. Открытые (public) свойства – это характеристики самолета (тип, серийный номер), наследуются дочерними классами пассажирских и грузовых самолетов.

Конструктор по умолчанию без параметров нужен для автоматического создания объектов десериализатором XML/JSON, кроме того, все свойства также должны иметь сеттеры (set) для автоматической установки значений.

Далее допишем дочерние классы, расширяющие базовый класс дополнительными свойствами: количеством пассажиров и весом груза. В дочерних классах необходимо переопределить абстрактный метод расчета взлетного веса. Для пассажирского самолета вес груза равен количеству пассажиров, умноженному на средний вес 62 кг.

```
/// <summary>
/// Пассажирский самолет
/// </summary>
public class PassengerPlane : Plane
{
    private const double K = 62;

    public PassengerPlane() { }

    public PassengerPlane(PlaneType type, string number, int count)
        : base(type, number)
    {
        Count = count;
    }

    /// <summary>
    /// Количество пассажиров
    /// </summary>
    public int Count { get; set; }

    public override double TakeoffWeight
    {
        get { return K*Count + _emptyWeight; }
    }
}
```

Конструктор дочернего класса делегирует установку части параметров конструктору базового класса. Конструктор по умолчанию нужен для автоматического создания объектов, его явно не нужно указывать только, если нет других конструкторов.

```
/// <summary>
/// Грузовой самолет
/// </summary>
public class CargoAircraft : Plane
{
    public CargoAircraft() { }

    public CargoAircraft(PlaneType type, string number, double weight)
        : base(type, number)
    {
        CargoWeight = weight;
    }

    /// <summary>
    /// Вес груза
    /// </summary>
    public double CargoWeight { get; set; }

    public override double TakeoffWeight
    {
        get { return CargoWeight + _emptyWeight; }
    }
}
```

Далее спроектируем класс-контейнер самолетов, Авиакомпанию, содержащую список объектов базового класса (List<Plane>), под которыми будут подразумеваться объекты дочерних классов. В качестве примера работы со списком, реализуем метод сортировки самолетов по взлетному весу.

```
/// <summary>
/// Авиакомпания
/// </summary>
public class Airline
{
    private readonly List<Plane> _planes = new List<Plane>();

    /// <summary>
    /// Суммарный вес всех самолетов
    /// </summary>
    public double TotalWeight
    {
        get
        {
            double weight = 0;

            foreach (var plane in _planes)
                weight += plane.TakeoffWeight;

            return weight;
        }
    }

    /// <summary>
    /// Добавить самолет
    /// </summary>
    public void Add(Plane plane)
    {
        if (plane == null || plane.Type == PlaneType.None
            || string.IsNullOrEmpty(plane.Number))
        {
            throw new ArgumentException(nameof(plane));
        }

        _planes.Add(plane);
    }
```

Вернем из класса список самолетов, но чтобы не нарушать целостность внутренней структуры данных, достаточно вернуть интерфейс перечисления IEnumerable<>, т.е. коллекцию только для просмотра (readonly).

```
/// <summary>
/// Выдать список самолетов
/// </summary>
public IEnumerable<Plane> GetPlanes()
{
    return _planes;
}
```

Сортировка списка самолетов по взлетному весу реализуется стандартным методом списка Sort() с передачей в него компаратора.

```
private class ByWeightComparer : IComparer<Plane>
{
    public int Compare(Plane x, Plane y)
    {
        return x.TakeoffWeight.CompareTo(y.TakeoffWeight);
    }
}

/// <summary>
/// Отсортировать по взлетному весу
/// </summary>
public void SortByWeight()
{
    _planes.Sort(new ByWeightComparer());
}
```

Допишем класс, добавив в него методы сериализации (сохранения) объектов-самолетов в XML-файл и десериализации (восстановления) объектов-самолетов из XML-файла. Но восстанавливать из файла удобно целиком объект-авиакомпанию, потому что кроме списка самолетов, у нее нет других данных.

```
    /// <summary>
    /// Сериализовать в XML-файл
    /// </summary>
    public void ToXml(string fileName)
    {
        var serializer = new XmlSerializer(typeof(List<Plane>));

        using (var stream = File.OpenWrite(fileName))
        {
            serializer.Serialize(stream, _planes);
            stream.Flush();
        }
    }

    /// <summary>
    /// Десериализовать из XML-файла
    /// </summary>
    public static Airline FromXml(string fileName)
    {
        var airline = new Airline();
        var serializer = new XmlSerializer(typeof(List<Plane>));

        using (var stream = File.OpenRead(fileName))
        {
            var planes = serializer.Deserialize(stream) as IEnumerable<Plane>;
            if (planes != null) airline._planes.AddRange(planes);
        }

        return airline;
    }
}
```

У операций сериализации/десериализации есть особенность, невозможно определить реальный тип объекта, если передавать сериализатору список ссылок на базовый класс. Поэтому, в C# придуман механизм маркировки типов и содержимого типов данных специальными атрибутами или метаданными, т.е. данными, описывающими программный код.

Атрибуты широко применяются в различных библиотеках и фреймворках, использующих рефлексию, т.е. механизм программного анализа кода. Благодаря атрибутам можно настроить некоторые аспекты сериализации, например, изменить имена тегов внутри XML-файла, настроить обработку null-значений свойств и т.д.

Добавим к базовому классу Plane атрибуты, дающие сериализатору информацию о наличие дочерних классов.

```
/// <summary>
/// Базовый абстрактный класс 'Самолет'
/// </summary>
[XmlInclude(typeof(PassengerPlane))]
[XmlInclude(typeof(CargoAircraft))]
public abstract class Plane
```

…

Синтаксически атрибут, являющийся объектом, заключается в квадратные скобки с указанием типа атрибута, параметров конструктора типа атрибута и необязательных свойств.

Такие темы, как атрибуты и рефлексия, выходят за пределы данного курса по C#.

Далее приведен пример использования выше описанных классов.

```
var myAirline = new Airline();

myAirline.Add(new PassengerPlane(PlaneType.AirbusA310, "RA-00001", 120));
myAirline.Add(new CargoAircraft(PlaneType.Boeing747, "RA-00002", 50000));
myAirline.Add(new PassengerPlane(PlaneType.Boeing737, "RF-00003", 80));

myAirline.SortByWeight();

const string fileName = @"c:\Temp\MyAirline.xml";
myAirline.ToXml(fileName);

try
{
    var myAirlineNew = Airline.FromXml(fileName);

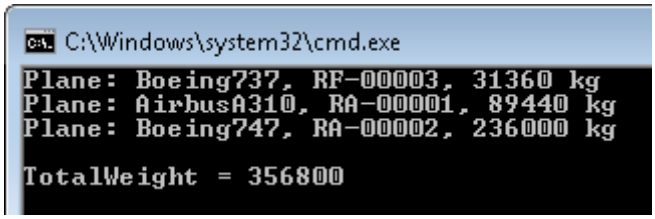
    foreach (var plane in myAirlineNew.GetPlanes())
    {
        Console.WriteLine(
            $"Plane: {plane.Type}, {plane.Number}, {plane.TakeoffWeight} kg");
    }

    Console.WriteLine($"\nTotalWeight = {myAirlineNew.TotalWeight}");
}
catch (Exception ex) { Console.WriteLine($"Error: {ex.Message}"); }
```

В коде создается объект-авиакомпания myAirline, в нее добавляются три самолета разных типов. После добавления внутренний список сортируется по взлетному весу методом SortByWeight().

Авиакомпания сохраняется в XML-файл, а затем восстанавливается из него в качестве нового объекта myAirlineNew. Десериализатор создает список самолетов, запускает дефолтные конструкторы и устанавливает значения всех свойств. Далее этот список добавляется к пустому списку новой авиакомпании. Удобно метод десериализации FromXml() сделать статическим, в таком случае он аналогичен конструктору.

Пример работы.



Пример XML-файла.

```
<?xml version="1.0"?>
- <ArrayOfPlane xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
    - <Plane xsi:type="PassengerPlane">
        <Type>Boeing737</Type>
        <Number>RF-00003</Number>
        <Count>80</Count>
      </Plane>
    - <Plane xsi:type="PassengerPlane">
        <Type>AirbusA310</Type>
        <Number>RA-00001</Number>
        <Count>120</Count>
      </Plane>
    - <Plane xsi:type="CargoAircraft">
        <Type>Boeing747</Type>
        <Number>RA-00002</Number>
        <CargoWeight>50000</CargoWeight>
      </Plane>
  </ArrayOfPlane>
```

Из особенностей, в файле для каждого объекта типа Plane есть специальный атрибут type, поясняющий реальный тип объекта. Пространства имен xsd и xsi создаются автоматически и несут служебную информацию.

Для работы с форматом JSON есть стандартные классы, например, **DataContractJsonSerializer**. Но удобнее взять популярную библиотеку **Json.NET** (https://www.newtonsoft.com/json). Документация к библиотеке доступна по ссылке https://www.newtonsoft.com/json/help/html/Introduction.htm. Интересует способ явно задать информацию о типе данных объекта в JSON (TypeNameHandling setting).

Реализуем в классе авиакомпании методы по работе с JSON, аналогичные XML. Для краткости, работать будем не в контексте потоков, а со строками.

```
/// <summary>
/// Сериализовать в JSON-файл
/// </summary>
public void ToJson(string fileName)
{
    File.WriteAllText(
        fileName, JsonConvert.SerializeObject(
            _planes, Formatting.Indented, new JsonSerializerSettings
                {
                    TypeNameHandling = TypeNameHandling.All
                }));
}

/// <summary>
/// Десериализовать из JSON-файла
/// </summary>
public static Airline FromJson(string fileName)
{
    var airline = new Airline();

    var planes = JsonConvert.DeserializeObject<List<Plane>>(
        File.ReadAllText(fileName), new JsonSerializerSettings
            {
                TypeNameHandling = TypeNameHandling.All
            });

    if (planes != null) airline._planes.AddRange(planes);
    return airline;
}
```

Пример файла MyAirline.json.


```
{
  "$type": "System.Collections.Generic.List`1[[ConsoleApplication.Plane, ConsoleApplication]], mscorlib",
  "$values": [
    {
      "$type": "ConsoleApplication.PassengerPlane, ConsoleApplication",
      "Count": 80,
      "TakeoffWeight": 31360.0,
      "Type": 3,
      "Number": "RF-00003"
    },
    {
      "$type": "ConsoleApplication.PassengerPlane, ConsoleApplication",
      "Count": 120,
      "TakeoffWeight": 89440.0,
      "Type": 1,
      "Number": "RA-00001"
    },
    {
      "$type": "ConsoleApplication.CargoAircraft, ConsoleApplication",
      "CargoWeight": 50000.0,
      "TakeoffWeight": 236000.0,
      "Type": 4,
      "Number": "RA-00002"
    }
  ]
}
```


В отличие от XML, в JSON попало свойство TakeoffWeight, хотя оно не имеет сеттера и не обязательно. Настройка попадания в JSON осуществляется атрибутами свойств класса, если это нужно. Имя типа данных ($type) представлено в виде длинной строки, это т.н. полное

квалифицированное имя класса, данное платформой .NET, включающее полное имя с пространством имен и, через запятую, имя сборки (dll или exe).

Все тонкости сериализации в XML и JSON описаны в документации к соответствующим библиотекам.